LIBRARY MANAGEMENT SYSYTEM USING PYTHON

Mini Project Report:

Submitted by

KALLURI NANDAKI

B.Tech – Computer Science and Engineering

Department of Computer Science and Engineering

RISE Krishna Sai Prakasam Group of Institutions

Academic Year: 2025–2026

CERTIFICATE

This is to certify that Mr./Ms. KALLURI NANDAKI, a student of B.Tech in Computer Science and Engineering, has successfully completed the mini project titled "Library Management System Using Python" during the academic year 2025–2026.The Department of Computer Science and Engineering.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their valuable guidance and support throughout the completion of this project.

I also thank my college for providing the necessary facilities and resources required for this project. Finally, I would like to thank my family and friends for their encouragement and support.

ABSTRACT

The Library Management System is a Python-based application developed to simplify library operations and maintain book records digitally. The system allows users to add, search, issue, return, view, and delete books efficiently.

The project also includes a login system for secure access and file handling to store book information permanently. The system reduces manual work, improves accuracy, and helps in managing library records in an organized manner.

This project is developed using Object-Oriented Programming concepts and provides a simple menu-driven interface for users.

INTRODUCTION

Managing library records manually can be difficult and time-consuming. Maintaining information about available books, issued books, and returned books using registers often leads to errors and data loss.

The Library Management System is developed using Python to simplify these tasks. The system provides a menu-driven interface that allows users to perform various library operations easily and efficiently.

The application stores book details in a text file, which helps in maintaining records even after the program is closed.

PROPOSED SOLUTION

To overcome the limitations of manual record management, a Library Management System is developed using Python.

The system provides features such as login authentication, adding books, viewing books, searching books, issuing books, returning books, deleting books, and displaying the total number of books.

File handling is used to store book records permanently, ensuring that data is available whenever the application is reopened.

TECHNOLOGIES USED

- Python
- Object-Oriented Programming (OOP)
- Functions
- Lists and Dictionaries
- File Handling
- Conditional Statements
- Loops
- Text File Storage

SYSTEM DESIGN

User → Login System → Library Management System → Book Operations → Book Records

The user logs into the system using a username and password. After successful authentication, the user can perform various operations such as adding, searching, issuing, returning, and deleting books.

MODULES

Login Module:

This module verifies the username and password before allowing access to the system.

Add Book Module:

This module is used to add new books with details such as book ID, book name, and author name.

View Books Module:

This module displays all available books along with their current status.

Search Book Module:

This module helps users search for a book using its unique book ID.

Issue Book Module:

This module is used to issue books to students and store their details.

Return Book Module:

This module updates the status of issued books when they are returned.

Delete Book Module:

This module removes unwanted or outdated book records from the system.

Total Books Module:

This module displays the total number of books available in the library.

ALGORITHM

1. Start the program.
2. Display the login screen.
3. Verify the username and password.
4. Load book records from the file.
5. Display the menu options.
6. Read the user's choice.
7. Perform the selected operation.
8. Update the records in the file.
9. Return to the main menu.
10. Exit the program.

PYTHON CODE

```
# ============================
# LIBRARY MANAGEMENT SYSTEM
# ============================

class Library:

    def __init__(self):
        self.books = []
        self.load_books()

    # Load books from file
    def load_books(self):
        try:
            with open("books.txt", "r") as f:
                for line in f:
                    data = line.strip().split(",")

                    if len(data) == 6:
                        book = {
                            "id": data[0],
                            "name": data[1],
                            "author": data[2],
                            "issued": data[3] == "True",
                            "student": data[4],
                            "roll": data[5]
                        }

                        self.books.append(book)

        except FileNotFoundError:
            pass

    # Save books to file
    def save_books(self):
        with open("books.txt", "w") as f:
            for book in self.books:
```

```
                f.write(
                    f"{book['id']},{book['name']},{book['author']},"
f"{book['issued']},{book['student']},{book['roll']}\n"
                )

    # Add Book
    def add_book(self):

        book_id = input("Enter Book ID: ")
        book_name = input("Enter Book Name: ")
        author = input("Enter Author Name: ")

        book = {
            "id": book_id,
            "name": book_name,
            "author": author,
            "issued": False,
            "student": "",
            "roll": ""
        }

        self.books.append(book)
        self.save_books()

        print("Book Added Successfully!")

    # View Books
    def view_books(self):

        if len(self.books) == 0:
            print("No Books Available")
            return

        print("\n===== BOOK LIST =====")

        for book in self.books:

            status = "Issued" if book["issued"] else "Available"
```

```
            print(
                f"ID: {book['id']} | "
                f"Name: {book['name']} | "
                f"Author: {book['author']} | "
                f"Status: {status}"
            )

    # Search Book
    def search_book(self):

        book_id = input("Enter Book ID: ")

        for book in self.books:

            if book["id"] == book_id:

                print("\nBook Found")
                print("Book ID :", book["id"])
                print("Book Name :", book["name"])
                print("Author :", book["author"])

                return

        print("Book Not Found")

    # Issue Book
    def issue_book(self):

        book_id = input("Enter Book ID: ")

        for book in self.books:

            if book["id"] == book_id:

                if not book["issued"]:

                    student = input("Enter Student Name: ")
                    roll = input("Enter Roll Number: ")
```

```
                book["issued"] = True
                book["student"] = student
                book["roll"] = roll

                self.save_books()

                print("Book Issued Successfully")

            else:
                print("Book Already Issued")

            return

    print("Book Not Found")

# Return Book
def return_book(self):

    book_id = input("Enter Book ID: ")

    for book in self.books:

        if book["id"] == book_id:

            if book["issued"]:

                book["issued"] = False
                book["student"] = ""
                book["roll"] = ""

                self.save_books()

                print("Book Returned Successfully")

            else:
                print("Book Was Not Issued")

            return
```

```
        print("Book Not Found")

    # Delete Book
    def delete_book(self):

        book_id = input("Enter Book ID: ")

        for book in self.books:

            if book["id"] == book_id:

                self.books.remove(book)
                self.save_books()

                print("Book Deleted Successfully")

                return

        print("Book Not Found")

    # Total Books
    def total_books(self):

        print("Total Books:", len(self.books))


# ============================
# LOGIN SYSTEM
# ============================

print("===== LIBRARY LOGIN =====")

saved_username = input("Create Username: ")
saved_password = input("Create Password: ")

print("\nAccount Created Successfully!")

username = input("\nEnter Username: ")
```

```
password = input("Enter Password: ")

if username == saved_username and password ==
saved_password:
    print("Login Successful")
else:
    print("Invalid Username or Password")
    exit()


# ============================
# MAIN PROGRAM
# ============================

library = Library()

while True:

    print("\n===== LIBRARY MANAGEMENT SYSTEM =====")
    print("1. Add Book")
    print("2. View Books")
    print("3. Search Book")
    print("4. Issue Book")
    print("5. Return Book")
    print("6. Delete Book")
    print("7. Total Books")
    print("8. Exit")

    choice = input("Enter Choice: ")

    if choice == "1":
        library.add_book()

    elif choice == "2":
        library.view_books()

    elif choice == "3":
        library.search_book()
```

```
elif choice == "4":
    library.issue_book()

elif choice == "5":
    library.return_book()

elif choice == "6":
    library.delete_book()

elif choice == "7":
    library.total_books()

elif choice == "8":
    print("Thank You")
    break

else:
    print("Invalid Choice")
```

CONCLUSION

The Library Management System successfully manages library records in a simple and efficient manner. The system reduces manual work and helps maintain accurate book information.

The project demonstrates the use of Python programming concepts such as object-oriented programming, file handling, loops, and conditional statements to solve a real-world problem.

FUTURE ENHANCEMENTS

- Add a graphical user interface using Tkinter.
- Connect the system with a MySQL database.
- Add fine calculation for late returns.
- Generate reports automatically.
- Implement role-based login for librarians and students.
- Add barcode scanning functionality.

REFERENCES

1. Python Official Documentation – https://docs.python.org/

2. W3Schools Python Tutorial – https://www.w3schools.com/python/

3. GeeksforGeeks Python Articles – https://www.geeksforgeeks.org/python-programming-language/

4. Python Crash Course by Eric Matthes.